Agentic AI

& Fundamentals

Week 2 · Turing × UCSB × ACM

Irene Fan & Agnibha Sarkar | March 6, 2026

Course Schedule

W1	Course Kickoff + Problem Scoping
W2	**Agentic AI & Fundamentals ← YOU ARE HERE**
W3	LangGraph & RAG
W4	HITL Systems
W5	Industry Realities of Building AI Products
W6	Architecture Design & Eng Design Doc
W7	Architecture Presentations + Critique
W8	Coding Copilots & Dev Cycle Integration
W9	End-to-End Implementation & Evaluation
W10	Demo Week

Today

1. What is Agentic AI?
2. Degrees of Autonomy
3. Why Go Agentic?
4. Task Decomposition
5. The Four Design Patterns
6. Deep Dive: Reflection
7. Evaluations
8. Hands-On Demo

How Do You Solve a Hard Problem?

Think about writing a cover letter for a job you really want.

Do you write it in one shot and hit send?

✗ **Nobody does that.**

One prompt. No revision. No feedback. Press send.

That's what most LLM apps do today.

More likely:

1. Research the company
2. Outline key points
3. Write a draft
4. Re-read, fix the tone
5. Ask someone to review
6. Revise and send

We iterate. We use tools. We get feedback. We revise. — So should our AI systems.

What Makes an AI System "Agentic"?

An AI agent is an LLM-powered system that can:

Plan

Decide what steps to take to complete a complex goal — not just respond to a single prompt.

Use Tools

Call APIs, query databases, execute code, search the web — interact with the world beyond text.

Remember

Carry context across multiple steps — not just one-shot with no memory between calls.

Reflect

Check its own work, critique its outputs, and improve based on feedback or external signals.

CBRE: One-Shot vs. Agentic

The same task — very different approaches.

ONE-SHOT

Give the LLM a transcript. Ask it to classify, assign severity, and route — all in one prompt.

One chance. No context. No validation.

AGENTIC

1. **Extract** entities from the transcript
2. **Look up** similar past problems in a database
3. **Classify** using extracted info + retrieved context
4. **Check** confidence — does it need human review?
5. **Route** to the right team

Degrees of Autonomy

"Agentic" isn't binary — it's a spectrum.

Low Autonomy	Semi-Autonomous	Highly Autonomous
• All steps predetermined • Tool use is hard-coded • AI only generates text • Fixed pipeline — no branching	• Agent picks which tools to use • All tools are predefined • Agent chooses when/whether to use them • Some decision-making freedom	• Many independent decisions • Can create new tools on the fly • Plans and adapts without being told how • Minimal human guidance

💬 **Discussion: Where should the CBRE system sit on this spectrum?**

Why Go Agentic?

>90%

Accuracy on HumanEval

Better Performance

GPT-3.5 + agentic patterns beats GPT-4 one-shot (~67%). A cheaper model with a good workflow outperforms a stronger model used naively.

50×

Throughput

Parallelism & Speed

Process 50 transcripts simultaneously. Query multiple databases in parallel. Hundreds of calls triaged while a human handles one.

LEGO Architecture

Modularity

Swap gpt-4o-mini for gpt-4o in one step. Replace ChromaDB with Pinecone. Add a weather API for HVAC — no redesign needed.

What Tasks Suit Agentic AI?

✓ Easier	⚠ Harder	CBRE sits in the middle
Clear, step-by-step process	Steps not known ahead of time	Steps partly predictable
Standard procedures to follow	Plan / solve as you go	Decisions depend on transcript
Text-only assets	Multimodal (sound, vision)	Text-based ✓

CBRE falls in the middle — steps are somewhat predictable, but decisions within each step depend on the content of the transcript.

Task Decomposition

The most important skill in building agentic systems.

For each step, ask: Is this an LLM, a Tool, or just Code?

LLM	Tool	Code
Requires language understanding or generation e.g. extract entities, classify, summarize	Database query, API call, or code execution e.g. vector search, webhook, rule engine	Simple deterministic logic (if/else, thresholds) e.g. check if confidence > 0.80

CBRE Pipeline — Decomposed

Step	Task	Handler
1	Extract problem type, location, severity cues	LLM
2	Look up matching problem codes	Tool (vector search)
3	Assign severity level	LLM
4	Check confidence score > 0.80?	Code
5	Route or flag for human review	Code

Group Exercise + Real-World Complexity

⏱️ 3-Minute Group Exercise

"A tenant calls and says there's a strong gas smell on the 3rd floor."

Walk through the steps your system should take.

For each step, label it:
- LLM — language understanding
- Tool — database / API / code execution
- Code — simple logic (if/else)

Not All Workflows Are Predictable

IT Helpdesk Agent:

1. Read issue description	LLM
2. Check knowledge base for match	Tool
3. If match → provide fix	LLM
4. If no match → ask clarifying question → loop back	LLM
5. If fix didn't work → escalate + create ticket	Tool

Which steps are needed depends on what happens at each step.

The Four Design Patterns

1. Reflection

Agent reviews its own output and improves it — like re-reading a draft before hitting send.

2. Tool Use

LLM calls external tools: APIs, databases, code execution. How agents interact with the world.

3. Planning

Agent decides what to do next based on context — instead of a hard-coded sequence.

4. Multi-Agent

Multiple specialized agents collaborate — one extracts, one classifies, one validates.

In our project → Reflection: today · Tool Use: today + W3 · Planning: W3 (LangGraph) · Multi-Agent: later

Deep Dive: Reflection

Instead of treating the first output as final, add a feedback loop.

Two Flavors

Self-Reflection

```
LLM generates → LLM critiques → LLM improves
```

Easier to set up. Limited to what the model already knows.

Reflection + External Feedback

```
LLM generates → Validator checks → Feed back
→ LLM fixes
```

More powerful — grounded in real data, not just re-reading itself.

Does Reflection Actually Work?

Self-Refine (Madaan et al., 2023)

Tested across 7 tasks: sentiment reversal, dialogue, code optimization, code readability, math reasoning, acronym generation, constrained generation.

Where Reflection Helps Most

Task	Problem Without Reflection	What Reflection Checks
Entity extraction	Misses compound hazards	"Safety risks underestimated?"
Structured data	Missing / malformed fields	Validate against schema
Summaries	Missing key details	"Includes problem, location, severity?"
Severity classification	Defaults to Medium	"Do urgency indicators match?"

Reflection in Practice

CBRE example + using different models + prompt tips

CBRE: Water Leak Example

"Water coming through ceiling, 4th floor, above server room. Getting worse."

Without Reflection

Problem: water leak · Severity: Medium
❌ Misses: water + server room = critical hazard

With Reflection

1. First pass: water leak, Medium
2. Reflect: "Check compound hazards"
3. ✓ Revised: Critical (water near servers)

Use Different Models

Generator (gpt-4o-mini) → cheaper, faster
Critic (gpt-4o) → stronger, checks edge cases

Cheaper model generates. Stronger model reviews.

Prompt Tips

1. Use "reflect" / "review" — not just "respond"
2. Give specific criteria — what to check for
3. Feed in the original output
4. Use external signals — rule checks beat self-review
5. Try different models — reasoning models critique well

Evaluations

The #1 predictor of success: a disciplined evaluation process. Measure → test → iterate → repeat.

Objective Evals

- Clear right/wrong answers
- e.g. correct severity label
- Accuracy = # correct / total
- This is what you'll do in the notebook

Example

TX-002: Predicted Medium → True Critical → ✗

Subjective Evals

- No single right answer
- Binary rubric criteria (yes/no)
- Does it mention problem type?
- Does it include the building name?

Example

"Is it one sentence as requested?" → yes/no

LLM-as-Judge

- Second LLM evaluates at scale
- Give it a rubric and let it score
- Not perfect — but scales to hundreds
- Works well for open-ended outputs

Example

GPT-4 critiques GPT-3.5 output on 500 transcripts

The Loop: Evals reveal what's broken → Fix prompt / pipeline → Re-evaluate → Repeat

Framework Landscape

We're using LangChain + LangGraph. Today: LangChain. Week 3: LangGraph.

Using now

LangChain

LLM abstraction layer — structured output, prompt templates, chains, parsers. The plumbing that connects prompts to models to output formats.

Week 3

LangGraph

Workflow orchestration on top of LangChain — graphs, state machines, conditional routing, loops. How you build multi-step pipelines.

FYI

CrewAI

Multi-agent role collaboration — define agents with specific roles, goals, and tools that work together on tasks.

FYI

AutoGen

Multi-agent conversations from Microsoft — agents that converse with each other to solve tasks iteratively.

Assignment

~3 hours total (setup, learning, exploring)

Part 1 **Think Like a Builder · ~45 min**

- 1a. Decompose your CBRE pipeline: label each step LLM / Tool / Code (4-5+ steps)
- 1b. Pick 1-2 human-in-the-loop checkpoints — explain what could go wrong without them
- 1c. Write one tricky transcript (~3-5 sentences) that would be hard for an AI to classify

Part 2 **Hands-On Notebook · ~1.5–2 hrs**

- Run entity_extraction_demo.ipynb end-to-end — note your baseline accuracy
- Improve the prompt (severity guidelines etc.) — re-run and compare accuracy
- Add a new Pydantic field; test your tricky transcript from 1c

Bring to Week 3: pipeline decomposition · accuracy changes · tricky transcript results

QUIZ

Check Your Understanding

See the quiz document

Recap

Agentic AI	Iterative, not one-shot
Autonomy	A spectrum — pick the right level
Benefits	Better performance, parallelism, modularity
Decomposition	LLM vs. Tool vs. Code for each step
Design Patterns	Reflection, Tool Use, Planning, Multi-Agent
Reflection	Review loop — catches missed mistakes
Evaluations	Measure, iterate, repeat

NEXT WEEK

Week 3: LangGraph & RAG

- Connect extraction to a problem code database
- Build a real multi-step pipeline
- LangGraph for conditional routing

Demo: entity_extraction_demo.ipynb → Transcript → Entity Extraction → Eval → Reflection